



DCS 2026 Spring HW02 – Code Review

TA : 陳孟頴 Date : 2026.04.07

Outline

- HW Review
- Latency Optimization
- Area Optimization
- FAQ

HW02: Sobel Edge Detection

- Algorithm:
 - Perform convolution-like operation on the input 8x8 image sequentially.
 - Compare the magnitude with threshold.

G_x			
	-1	0	1
	-2	0	2
	-1	0	1

input								
	156	155	98	30	203	205	133	137
	176	183	96	183	221	167	9	224
	108	23	178	227	110	248	110	38
	111	241	7	114	189	13	199	13
	121	65	140	195	159	215	121	167
	51	46	54	148	147	165	157	227
	43	217	44	148	4	43	51	165
	7	19	213	184	53	76	32	29

$$|G_x| = |156 \cdot -1 + 176 \cdot -2 + 108 \cdot -1 + 98 \cdot 1 + 96 \cdot 2 + 178 \cdot 1| = 148$$

$$|G_y| = |156 \cdot -1 + 155 \cdot -2 + 98 \cdot -1 + 108 \cdot 1 + 23 \cdot 2 + 178 \cdot 1| = 232$$

$$|G_x| + |G_y| > 128, \text{ out_data} = 1 \text{ (center pixel[1][1])}$$

G_y			
	-1	-2	-1
	0	0	0
	1	2	1

input								
	156	155	98	30	203	205	133	137
	176	183	96	183	221	167	9	224
	108	23	178	227	110	248	110	38
	111	241	7	114	189	13	199	13
	121	65	140	195	159	215	121	167
	51	46	54	148	147	165	157	227
	43	217	44	148	4	43	51	165
	7	19	213	184	53	76	32	29

$$|G_x| = |155 \cdot -1 + 183 \cdot -2 + 23 \cdot -1 + 30 \cdot 1 + 183 \cdot 2 + 227 \cdot 1| = 79$$

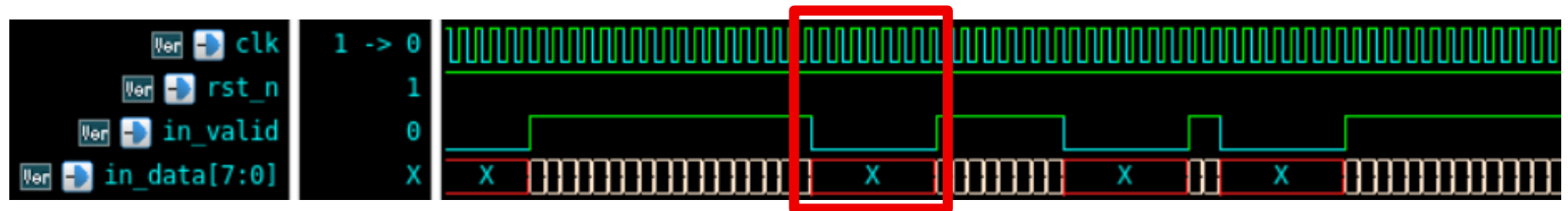
$$|G_y| = |155 \cdot -1 + 98 \cdot -2 + 30 \cdot -1 + 23 \cdot 1 + 178 \cdot 2 + 227 \cdot 1| = 225$$

$$|G_x| + |G_y| > 128, \text{ out_data} = 1 \text{ (center pixel[1][2])}$$

HW02: Sobel Edge Detection

- Specification:
 - Each input pixel has a 10% chance to stall, and every stall takes 8 cycles.
 - Nonconsecutive output is acceptable.

Randomly stall input



Nonconsecutive output

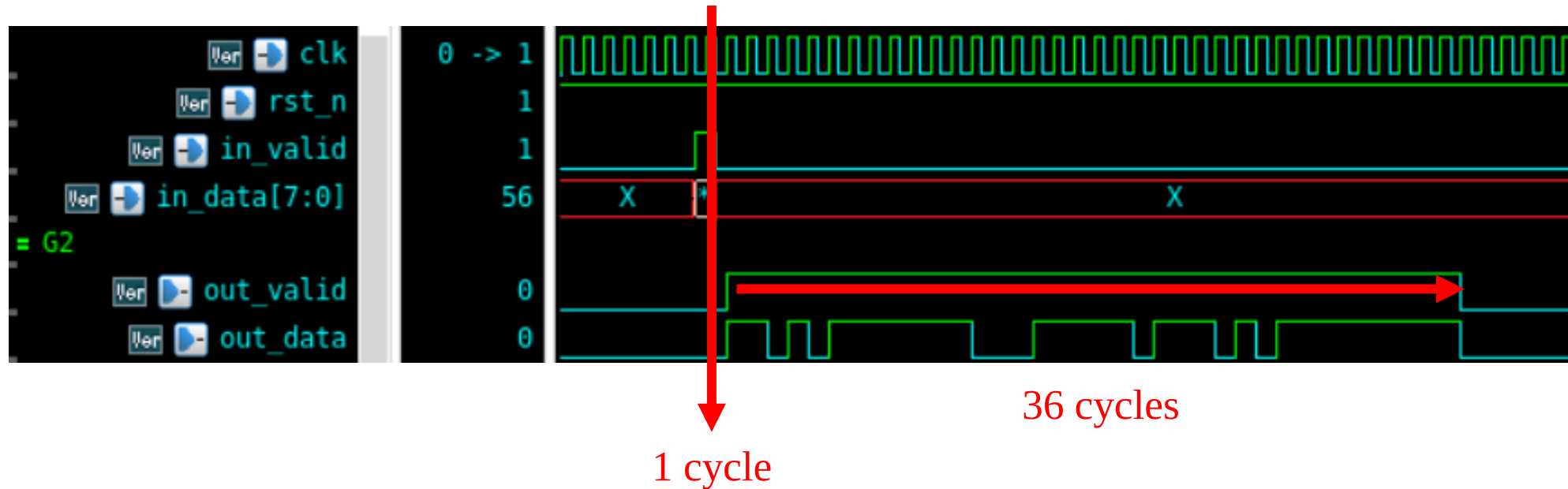


Outline

- HW Review
- Latency Optimization
- Area Optimization
- FAQ

Output Latency Optimization

- Naive Solution:
 - Output the result as soon as receiving all inputs and in_valid goes low.
 - Minimum latency = $1 + 36 = 37$ per pattern.



Output Latency Optimization

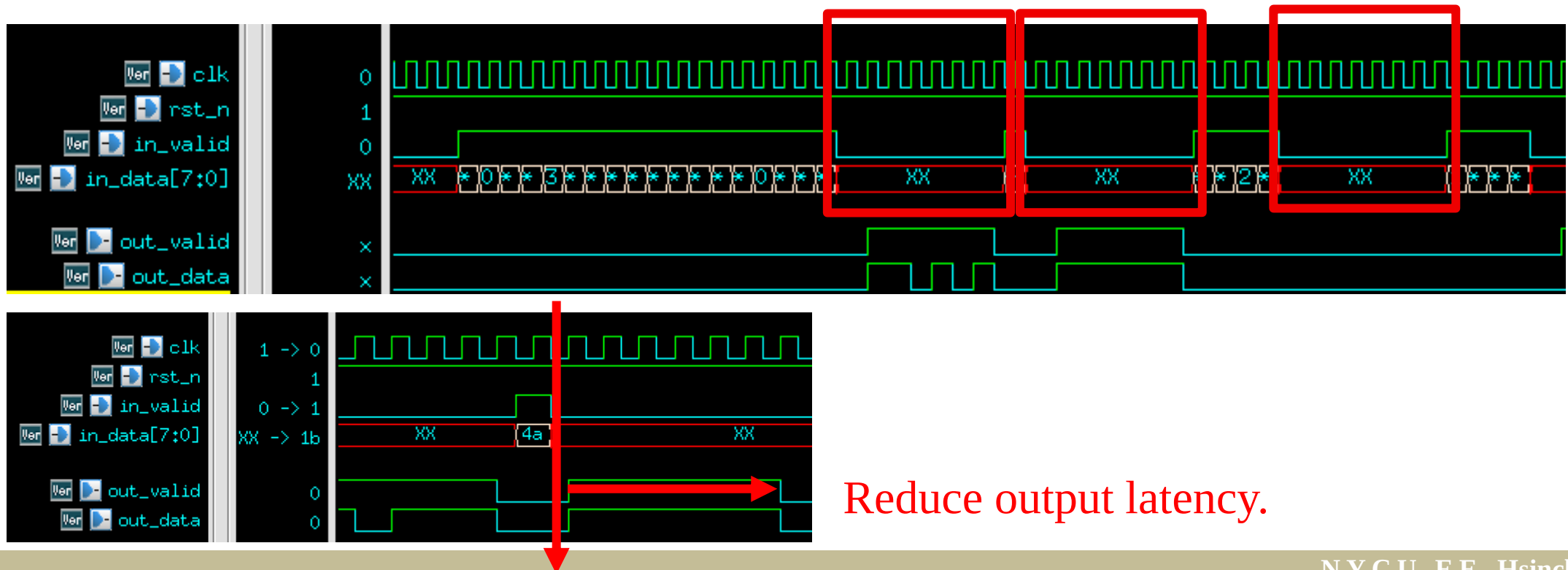
- Better Solution:

input								
	156	155	98	30	203	205	133	137
	176	182	96	182	221	167	9	224
	108	23	178	227	110	248	110	38
	111	241	7	114	189	13	199	13
	121	65	140	195	159	215	121	167
	51	46	54	148	147	165	157	227
	43	217	44	148	4	43	51	165
	7	19	213	184	53	76	32	29

We only need 19 inputs to
get first output result.

Output Latency Optimization

- Better Solution:
 - Calculate the output once sufficient input data is received.
 - Drive the output results when the in_valid signal is deasserted.



Output Latency Optimization

- Better Solution:
 - Calculate the output once sufficient input data is received.
 - Drive the output results when the in_valid signal is deasserted.
- Implementation:
 - FIFO
 - Register with pointer

Outline

- HW Review
- Latency Optimization
- Area Optimization
- FAQ

Area Optimization – Register Minimization

- Naive Solution:
 - Store all $8*8 = 64$ pixels.
- Better Solution:
 - Store needed 19 pixels.

input								
	156	155	98	30	203	205	133	137
	176	183	96	183	221	167	9	224
	108	23	178	227	110	248	110	38
	111	241	7	114	189	13	199	13
	121	65	140	195	159	215	121	167
	51	46	54	148	147	165	157	227
	43	217	44	148	4	43	51	165
	7	19	213	184	53	76	32	29

input								
	156	155	98	30	203	205	133	137
	176	183	96	183	221	167	9	224
	108	23	178	227	110	248	110	38
	111	241	7	114	189	13	199	13
	121	65	140	195	159	215	121	167
	51	46	54	148	147	165	157	227
	43	217	44	148	4	43	51	165
	7	19	213	184	53	76	32	29

Area Optimization – Avoid Multiplier

- Naive Solution:
 - $G_x = A*-1 + D*-2 + G*-1 + C*1 + F*2 + I*1$
 - $G_y = A*-1 + B*-2 + C*-1 + G*1 + H*2 + I*1$

G_x			
	-1	0	1
	-2	0	2
	-1	0	1

G_y			
	-1	-2	-1
	0	0	0
	1	2	1

input								
	A	B	C	30	203	205	133	137
	D	E	F	183	221	167	9	224
	G	H	I	227	110	248	110	38
	111	241	7	114	189	13	199	13
	121	65	140	195	159	215	121	167
	51	46	54	148	147	165	157	227
	43	217	44	148	4	43	51	165
	7	19	213	184	53	76	32	29

Area Optimization – Avoid Multiplier

- Naive Solution:
 - $G_x = A \cdot -1 + D \cdot -2 + G \cdot -1 + C \cdot 1 + F \cdot 2 + I \cdot 1$
 - $G_y = A \cdot -1 + B \cdot -2 + C \cdot -1 + G \cdot 1 + H \cdot 2 + I \cdot 1$
- Better Solution:
 - Use shift instead of multiplier for powers of 2.
 - $G_x = -A - (D \ll 1) - G + C + (F \ll 1) + I$
 - $G_y = -A - (B \ll 1) - C + G + (H \ll 1) + I$

Area Optimization – Algorithm Simplification

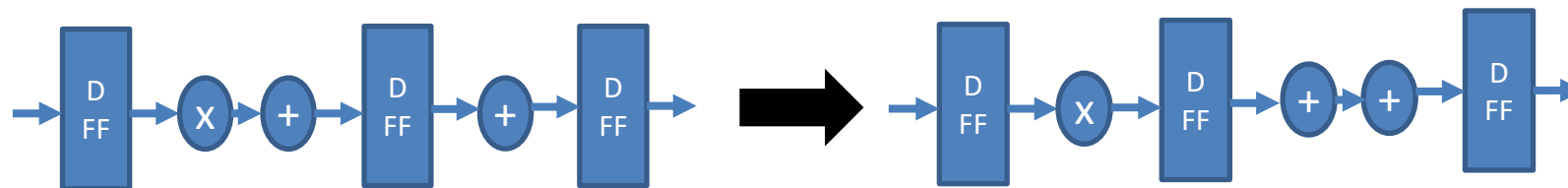
- Naive Solution:
 - $|Gx| + |Gy| > 128$
- Better Solutions:
 - Use logic gates instead of comparator
 - For [10:0] sum, assign $\text{edge_detected} = (|\text{sum}[10:8]) \mid (\text{sum}[7] \ \& \ (|\text{sum}[6:0]))$;
 - $|Gx| + |Gy| \rightarrow \max(|Gx + Gy|, |Gx - Gy|)$
 - Others

Outline

- HW Review
- Latency Optimization
- Area Optimization
- **FAQ**

Relationship between Timing & Area

- During 02_SYN synthesis, Design Compiler balances speed versus area.
 - **Sufficient Slack:** The compiler focuses on area minimization.
 - **Tight Timing:** The compiler swaps in larger, higher-performance cells to achieve timing closure.
- Reduce critical path for better performance.
 - Avoid multiple computation in single cycle.



Pass 01 Simulation but 03 Simulation Fail

- Hold Time Violation
 - Set the reset to a constant 0; do not use an input for the reset signal.

```
Warning! Timing violation
$hold( posedge SN:136228 FS, posedge RN:119954 FS, 0.486775 : 486775 FS );
File: /usr/cad/umc018/Verilog/umc18_neg.v, line = 9380
Scope: TESTBED.I_GCD.next_in_valid_reg
Time: 191897 FS
```

- Coding Style
 - Avoid multiple variables in single always block.

Good

```
always_comb begin
    outl_nxt = ct + 1;
end

always_ff @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        outl <= 0;
    end
    else begin
        outl <= outl_nxt;
    end
end
```

Bad

```
always_comb begin
    // GOT THE VALUE
    next = state;
    out_valid = 0;
    out_data = 0;
    ng = g;
    ngcd = gcd;
    nevenbefore = evenbefore;
    noddbefore = oddbefore;
    ncyclecount = cyclecount;

    if (rst_n) begin
        next = START;
    end else begin
        case(state)
            START: begin
                // state trans
                if(in_valid) begin
                    next = INPUT;
                end else begin
                    next = START;
                end

                // reset
                ncyclecount = 0;
                ng = 14'b0;
                ngcd = 0;
                nevenbefore = 0;
                noddbefore = 0;
            end

            INPUT: begin
                // start to input
                ncyclecount = ncyclecount + 1;
                case (indata[0])
```

Thank you.
Any questions?